

Guion de presentación

Sistema Experto de Recomendaciones Académicas

Cuatro bloques de exposición y el paso a paso de la demostración

Alineado con el Documento de Diseño y Arquitectura

Duración estimada: 15 minutos, más preguntas

Agosto de 2026

Cuatro bloques, alineados con el documento de diseño ([DISEÑO-Y-ARQUITECTURA.pdf](#)). Duración total estimada: **15 minutos**, más preguntas.

Las frases en cursiva son para decirlas tal cual si sirve. El resto es qué cubrir, no cómo.

Bloque	Tema	Minutos	Secciones del PDF
1	Contexto y problema	3	1 y 11
2	Decisiones de arquitectura	4	2, 4 y 5
3	Requerimientos y reglas	3	6 y 7
4	Demostración en vivo	5	3, 8, 9 y 10

Si son dos personas: una toma los bloques 1 y 2, la otra el 3 y el 4. El corte natural está entre «cómo lo diseñamos» y «qué hace».

Bloque 1 — Contexto y problema (3 min)

Objetivo: que quede claro qué problema resuelve y por qué era un sistema experto y no un programa cualquiera.

Qué decir

Arrancá con el problema, no con la tecnología.

«Cada cuatrimestre el estudiante tiene que cruzar a mano el plan de estudios, lo que ya aprobó, los choques de horario y hacia dónde quiere ir profesionalmente. Nadie tiene tiempo de hacerlo bien, y las consecuencias se ven tarde: semestres sobrecargados, y cursos que resultan ser cuello de botella y se descubren cuando ya atrasaron la carrera.»

Después el planteamiento del proyecto:

«Construimos un sistema experto en Common Lisp que recomienda cursos según intereses, cursos aprobados, horario disponible, dificultad y área profesional, y que además da estadísticas.»

Y ahora la frase que ordena toda la presentación:

«Lo importante no es que ordene una lista de cursos. Eso lo resolvíamos con un filtro y un `sort` en una tarde. Lo importante es que hay un motor de inferencia propio, con reglas declarativas, que además **explica** por qué recomienda cada curso.»

Cerrá el bloque con la honestidad de los datos

Esto conviene decirlo temprano y de frente. Si sale después, a pregunta, parece que lo estaban escondiendo.

«Un aviso sobre los datos: el programa que nos dio la universidad trae código, nombre, cuatrimestre, laboratorio y bloque electivo. No trae créditos, ni prerequisites, ni horarios. Todo eso lo asignamos nosotros con criterios documentados, y está marcado curso por curso en el archivo de datos. Así que el razonamiento del motor es real; los datos de entrada son parciales, y lo decimos.»

Bloque 2 — Decisiones de arquitectura (4 min)

Objetivo: mostrar que las decisiones tienen justificación escrita, no que salieron por casualidad.

2.1 Las tres capas

Mostrá el diagrama de la sección 2 del PDF.

«El sistema tiene tres capas y las dependencias van en una sola dirección. La CLI depende del dominio, el dominio depende del motor, y el motor no depende de nada.»

La regla que sostiene todo:

«El motor no puede saber qué es un curso. Ningún archivo de `engine/` menciona un curso, un crédito o un requisito, y su suite de pruebas usa hechos inventados como `(color rojo)`. Si el motor necesitara conocer el dominio para pasar sus pruebas, dejaría de ser un motor y sería un recomendador disfrazado.»

2.2 Por qué el motor es propio

La decisión que más van a preguntar. Está en ADR-005.

«Evaluamos usar CLIPS o LISA. Los descartamos a propósito: el trabajo se habría convertido en integración y el núcleo evaluable habría quedado fuera del repositorio.»

Si preguntan por RETE:

«También consideramos implementar una red RETE. La rechazamos por riesgo de cronograma: a esta escala, 47 cursos y 25 reglas, el matching directo resuelve en menos de un segundo. Está documentado como trabajo futuro.»

2.3 El conocimiento es dato, no código

Mostrá una `defrule` del PDF, sección 6.

«Las reglas no son código, son estructuras de datos que el motor interpreta. Agregar conocimiento nuevo es agregar una regla; no se toca el motor. Y eso no es una promesa: hay una prueba que agrega una regla en tiempo de ejecución y verifica que dispara sin modificar nada del motor.»

2.4 El modelo de hechos

Sección 5 del PDF, diagrama de las tres capas de hechos.

«Todo lo que el motor conoce es un hecho: una lista plana con la relación al frente. Hay tres capas: lo que viene del catálogo, lo que viene del perfil del estudiante, y lo que el motor deriva. Ninguna regla puede afirmar un hecho de las dos primeras. Esa disciplina es la que hace que la traza se pueda leer.»

Bloque 3 — Requerimientos y reglas (3 min)

Objetivo: mostrar que hay conocimiento experto de verdad, especificado antes de programarlo.

3.1 De requisito a regla

«Los requisitos están en un PRD, con requisitos funcionales y no funcionales. Cada pieza de conocimiento experto se especificó primero en lenguaje natural, con un identificador BR, y después se implementó. Cada regla del código cita en su docstring el BR que implementa, así que la especificación y el código se pueden confrontar en cualquier momento.»

Mostrá la tabla de la sección 7 del PDF.

3.2 Los tres tipos de regla

«Hay reglas duras, que son invariantes: nunca se recomienda un curso sin sus requisitos, ni uno que choque con el horario. Hay reglas estándar, como el cuello de botella. Y hay reglas blandas de priorización, que son los pesos: el área objetivo suma 10, un interés suma 5, ser cuello de botella suma 8.»

3.3 La regla que muestra conocimiento experto real

Esta es la que vale la pena contar despacio. Prepara la demo del bloque 4.

«La más interesante es una excepción. La regla dice que no se recomienda un curso que supere la tolerancia a la dificultad que declaró el estudiante. Pero si ese curso es cuello de botella y la supera por un solo nivel, sí se recomienda, con una advertencia: atrasarlo cuesta más que llevarlo.»

«Eso es conocimiento experto: una excepción con criterio, no un filtro rígido. Y lo van a ver funcionando ahora.»

Bloque 4 — Demostración en vivo (5 min)

Objetivo: que vean el sistema corriendo y entiendan lo que aparece en pantalla.

El recorrido completo, pantalla por pantalla y con lo que hay que decir en cada una, está al final de este guion: **«Paso a paso de la demostración»**. Ese es el que hay que llevar abierto el día de la presentación.

En resumen, el orden es:

1. El repositorio y la estructura del proyecto
2. Los datos, y qué parte es oficial
3. Una regla, para mostrar que el conocimiento es dato
4. El gate de verificación
5. Una sesión completa, sección por sección
6. La sesión interactiva con la excepción de dificultad
7. Las pruebas

Preguntas que van a hacer

Pregunta	Respuesta corta
¿Esto es un sistema experto o un filtro con nombre bonito?	Las pruebas de aceptación 4 y 5. El motor corre con (color rojo) sin un solo símbolo académico, y una regla declarada en tiempo de ejecución dispara sin tocar el motor.
¿Por qué no usaron CLIPS?	ADR-005. Habría convertido el trabajo en integración y sacado el núcleo evaluable del repositorio.
¿De dónde sacaron que este curso es dificultad 4?	Fórmula documentada: cuatrimestre más uno si tiene laboratorio, tope 5. Se deriva de dos datos que la universidad sí da, así que es reproducible. Es heurística del equipo, no clasificación de la universidad.
¿Estos prerrequisitos son reales?	No. El programa no declara ninguno; pusimos seis de demostración. Los cuellos de botella son correctos dado el grafo cargado , y ese grafo es de ejemplo. Está documentado campo por campo.
¿Y si consiguen los datos oficiales?	Es un cambio en data/ y cero líneas de código: la arquitectura mantiene los datos fuera del programa (ADR-006).
¿Cómo garantizan que no recomienda algo imposible?	Es un invariante probado sobre todos los perfiles: ninguna recomendación viola un requisito ni choca con el horario.
¿El motor puede quedarse en un bucle?	No. Refracción: una misma instanciación no dispara dos veces. Hay una prueba con una regla que se reactiva a sí misma y verifica que llega a quiescencia.
¿Por qué el tope de créditos no es una regla?	Compara entre sí un conjunto de tamaño variable, y el matching por patrones tiene aridad fija. Corre como post-procesamiento después de la quiescencia, y está documentado como limitación del modelo de matching.

Errores a evitar

- **No arranques por el código.** El problema primero, la solución después.
 - **No leas el PDF en voz alta.** El PDF es el respaldo; ustedes cuentan la historia.
 - **No escondas lo provisional.** Decilo en el bloque 1, con naturalidad. Si sale a pregunta, se ve peor.
 - **No prometas lo que no está.** No hay interfaz web, no hay matrícula real, no hay aprendizaje automático — y cada exclusión tiene su justificación.
 - **No corras la demo por primera vez en vivo.** Probala antes; la primera corrida compila y tarda.
-

Comandos, para tener a mano

```
$ sh .ace/scripts/verify.sh           # el gate: VERIFY_RESULT=pass
$ sbcl --script run.lisp               # sesión completa con perfil fijo
$ sbcl --script run-interactive.lisp   # sesión que pregunta (opción 6 para la demo)
$ sbcl --script run-tests.lisp         # 358 pruebas
```

Paso a paso de la demostración

Esto es lo que hay que llevar abierto el día de la presentación. Cada paso tiene **qué escribís**, **qué aparece** y **qué decís**.

Antes de empezar: terminal abierta en la raíz del repositorio, con la fuente grande. Corré todo una vez antes de presentar — la primera corrida compila y tarda, y en vivo esos segundos se hacen eternos.

Paso 0 — El repositorio (30 s)

Qué mostrás: la página del repositorio en GitHub.

«Bueno, acá tenemos el repositorio. Lo primero que se ve es el README, que explica qué es el proyecto y por dónde empezar a leer. Todo está en la rama principal.»

Bajá un poco y señalá la tabla de estado.

«Acá está el estado: las 14 tareas verificadas, 358 pruebas pasando y el gate de verificación en verde. Y estos son los documentos: el de diseño y arquitectura, la guía de ejecución y el informe.»

Paso 1 — La estructura del proyecto (45 s)

Qué escribís:

```
$ ls
```

«Nos vamos a la terminal. Esto es el proyecto por dentro.»

Señalá las tres carpetas que importan:

«`src` es el código fuente, `data` son los datos —el catálogo de cursos y los perfiles— y `docs` es toda la documentación. Estos tres archivos de arriba son los puntos de entrada: uno corre la demostración, otro la sesión interactiva y otro las pruebas.»

Qué escribís:

```
$ ls src/engine src/domain src/cli
```

«Y acá está la división en tres capas de la que hablamos. `engine` es el motor genérico: no sabe qué es un curso. `domain` es el conocimiento académico, las 25 reglas. Y `cli` es lo único que habla con el usuario.»

Paso 2 — Los datos, y qué parte es oficial (45 s)

Qué escribís:

```
$ head -40 data/courses.lisp
```

«Acá está el catálogo: 47 cursos reales del Bachillerato en Ingeniería en Sistemas de Computación. Y esta cabecera es importante.»

Señalá el bloque `OFICIAL` vs. `PROVISIONAL`.

«El programa que nos dio la universidad trae el código, el nombre, el cuatrimestre, si tiene laboratorio y a qué bloque electivo pertenece. Eso es oficial. Los créditos, la dificultad, el área y los prerrequisitos los asignamos nosotros, con el criterio que está escrito acá, y cada curso lo tiene marcado línea por línea.»

Qué escribís:

```
$ grep -A 12 "SC-304" data/courses.lisp | head -14
```

«Así se ve un curso. Fíjense que los datos no son código: son listas que el programa lee. Si mañana la universidad nos da los prerequisites reales, se cambian acá y no se toca ni una línea de código.»

Paso 3 — El conocimiento es dato (1 min)

Qué escribís:

```
$ grep -B 2 -A 8 "eligible-when-prerequisites" src/domain/knowledge.lisp
```

«Esta es una regla del sistema. No es una función: es una estructura de datos que el motor interpreta.»

Leé la regla en voz alta, señalando:

«Dice: si existe un curso, y no está aprobado, y sus requisitos están satisfechos, y su horario calza, entonces ese curso es elegible. El docstring cita BR-001, BR-002 y BR-003, que son las reglas de negocio que escribimos antes de programar.»

«Agregar conocimiento nuevo al sistema es agregar una de estas. El motor no se toca. Hay una prueba que lo verifica: agrega una regla en tiempo de ejecución y comprueba que dispara.»

Paso 4 — El gate de verificación (30 s)

Qué escribís:

```
$ sh .ace/scripts/verify.sh
```

Qué aparece, al final:

```
[ok] Gate 'test' passed.  
All configured gates passed.  
VERIFY_RESULT=pass gate=all
```

«Esto compila el sistema completo y corre las 358 pruebas. La última línea es el veredicto, pensado para leerse de un vistazo. Si dijera fail, algo está roto de verdad: el gate sale con error y no deja cerrar una tarea.»

Paso 5 — Una sesión completa, sección por sección (2 min)

Qué escribís:

```
$ sbcl --script run.lisp
```

«Esto corre una sesión completa con un perfil de ejemplo. Vamos a subir hasta arriba y verlo por partes.»

Subí al principio de la salida.

5.1 · PERFIL ANALIZADO

- Cursos aprobados: SC-202, SC-315, SC-103, SC-115, II-115
- Intereses: infrastructure, management, databases, software-engineering
- Tolerancia de dificultad: 4
- Area profesional objetivo: software-engineering

«Primero repite el perfil que analizó: qué aprobó el estudiante, qué le interesa, cuánta dificultad tolera y hacia dónde quiere ir.»

5.2 · RECOMENDACIONES

1. Estructura de Datos

Codigo: SC-304

Puntuacion: 27

Dificultad: 4

Razones:

- Es un curso cuello de botella: atrasarlo bloquea otros cursos (+8 puntos).
- Coincide con uno de tus intereses declarados (+5 puntos).
- Coincide con tu area profesional objetivo (+10 puntos).
- Abre camino a un curso de tu area profesional objetivo (+4 puntos).

«Acá está lo importante. No solo dice qué recomienda: dice **por qué**. Esas cuatro razones no son un texto fijo: cada una es una regla que efectivamente disparó sobre este curso. Y el puntaje 27 es la suma: 8 más 5 más 10 más 4. Cada sumando se puede rastrear hasta la regla que lo produjo.»

5.3 · CURSOS DESCARTADOS

- Introduccion al Calculo o Matematica Basica

Motivo: No cabe dentro de tu tope de creditos, dado el orden de prioridad.

«También explica lo que **no** recomendó, y con qué motivo. Muchas veces saber por qué un curso quedó fuera importa más que por qué otro entró.»

Mencioná que hay seis motivos distintos:

«Los motivos son seis: ya aprobado, le faltan requisitos, choca con el horario, es demasiado difícil, no cabe en los créditos, o ya hay otra electiva mejor puntuada del mismo bloque.»

5.4 · ESTADÍSTICAS

- Créditos aprobados: 20 de 188
- Avance de carrera: 10.6%
Avance relativo al catálogo modelado (47 cursos), no al plan de estudios completo de la carrera.
- Distribución de aprobados por área:

general-education	1 curso(s), 4 crédito(s)
mathematics	2 curso(s), 8 crédito(s)
software-engineering	2 curso(s), 8 crédito(s)

«Estas son las estadísticas del estudiante: cuánto lleva de la carrera, créditos y en qué áreas. Y fíjense en la aclaración: el avance es relativo al catálogo que modelamos, no a la carrera completa. Preferimos decirlo a que alguien se lleve una idea equivocada.»

5.5 · ESTADÍSTICAS DEL CATÁLOGO

Cursos cuello de botella (cuántos cursos desbloquean):
SC-304 Estructura de Datos 3 curso(s)

Cobertura de reglas: 22 de 25 dispararon

«Y estas son del catálogo, no del estudiante: qué cursos son cuello de botella, la dificultad promedio por área, y la cobertura de reglas.»

«Esa última la usamos durante el desarrollo: nos dice cuántas de las 25 reglas dispararon. Si una nunca dispara, o sobra o los datos no la ejercitan. Sirve para encontrar conocimiento muerto.»

5.6 · TRAZA DEL MOTOR

- Regla aplicada: schedule-block-unavailable-detected (ciclo 1)
Hecho generado: (schedule-block-unavailable SC-805)

«Y esto es el ciclo de inferencia completo, disparo por disparo. Son casi 300 en esta corrida. Esta es la evidencia de que hay un motor de verdad y no lógica cableada: de acá salen las explicaciones que vimos arriba, no las escribimos a mano.»

Paso 6 — La sesión interactiva y la excepción (1 min 30 s)

Qué escribís:

```
$ sbcl --script run-interactive.lisp
```

Qué aparece:

Como querés armar el perfil?

- 1) Respondiendo unas preguntas
- 2) Perfil de demostracion (ejercita las 6 razones de descarte)
- 3) Primer ingreso, sin cursos aprobados
- 4) Estudiante avanzado, quinto cuatrimestre cumplido
- 5) Trabaja: solo dos bloques libres por semana
- 6) Tolerancia baja a la dificultad

«El sistema también puede preguntarle el perfil al estudiante. La opción 1 hace seis preguntas; las otras cargan perfiles de ejemplo que preparamos para mostrar comportamientos distintos.»

Elegí la opción 6 y esperá el resultado.

«Voy a usar el de tolerancia baja, porque muestra la parte más interesante del sistema.»

Qué aparece:

1. Estructura de Datos

Puntuacion: 22

Dificultad: 4

Razones:

- Es un curso cuello de botella: atrasarlo bloquea otros cursos (+8 puntos).
- Coincide con tu area profesional objetivo (+10 puntos).
- Abre camino a un curso de tu area profesional objetivo (+4 puntos).

Advertencias:

- Supera tu tolerancia a la dificultad por un nivel, pero es un curso cuello de botella: atrasarlo cuesta mas que llevarlo ahora.

Pará acá. Es el momento fuerte de la demostración.

«Miren esto. Este estudiante declaró que tolera poca dificultad, y el sistema le está recomendando un curso más difícil de lo que pidió.»

Pausa.

«No es un error. Es una excepción que está en la base de conocimiento: si un curso es cuello de botella y supera la tolerancia por **un solo nivel**, se recomienda igual, porque atrasarlo cuesta más que llevarlo. Y el sistema lo dice: no lo esconde, lo advierte y explica por qué.»

«Eso es lo que separa un sistema experto de un filtro. Un filtro habría descartado el curso y el estudiante nunca se habría enterado.»

Paso 7 — Las pruebas (30 s)

Qué escribís:

```
$ sbcl --script run-tests.lisp
```

Qué aparece, al final:

```
Did 358 checks.  
  Pass: 358 (100%)  
  Skip: 0 ( 0%)  
  Fail: 0 ( 0%)
```

«358 comprobaciones. Entre ellas están los diez criterios de aceptación del proyecto: que ninguna recomendación viole un requisito ni choque con el horario, que el motor sea genérico, que se pueda agregar una regla sin tocarlo, que siempre termine, y que la misma entrada dé siempre la misma salida.»

Paso 8 — Cierre (30 s)

«Para cerrar: el sistema corre una sesión completa en menos de un segundo, tiene 25 reglas, 47 cursos reales y 358 pruebas en verde. Todo está en el repositorio, con el documento de diseño, la guía de ejecución y el informe.»

«Y lo que queremos que quede: no construimos un programa que ordena cursos. Construimos un motor de inferencia que razona sobre conocimiento declarado, y que puede explicar cada conclusión a la que llega.»

Si algo falla en vivo

Problema	Qué hacer
La primera corrida tarda mucho	Es la compilación. Por eso hay que correr todo una vez antes.
<code>sbc1: command not found</code>	Terminal equivocada, o SBCL no está en el PATH de esa sesión.
La sesión interactiva no responde	Se corrió con la entrada redirigida. Usar una terminal normal.
Sale un error de datos	El mensaje dice el archivo y el problema. Es el manejo de errores funcionando: mostralo como tal.

Regla general: si algo se rompe, no lo escondas. Decí qué pasó y seguí con el siguiente paso. Tenés el PDF y las pruebas como respaldo.